Milestone: 20GB Reached

We've reached 20GB of stored data — thank you for helping us grow!

Patreon Ko-fi Liberapay
Close
Freedium

- ko-fi
- librepay
- patreon

< Go to the original

Original Sin of Agents: Morris II, EchoLeak and Prompt Pond

We risk resurrecting the original sin of computing, a flaw that has enabled remote code execution exploits for decades.

Agent Native Follow

~11 min read · February 10, 2026 (Updated: February 10, 2026) · Free: No

Just last month, **Jaana Dogan** posted that **Claude Code** generated in an hour a distributed system her team spent a year building.

That's incredible but agents are getting powerful faster than our ability to secure them.

With most developers using AI coding assistants weekly and big companies deploying agent workflows, we need to talk about the part nobody puts in the demo: **prompt injection is RCE wearing a hoodie**.

In this article I will walk you through why agents inherit fundamental vulnerabilities, the exploit patterns already emerging, and the practical ways to set up agentic workflows that don't turn your machine into a liability.

> I'm writing a deep-dive eBook on **Agentic SaaS,** the emerging design patterns that are quietly powering the most innovative startups of 2026.

Grab the first chapter free: [Agentic SaaS Patterns Winning in 2026](), packed with real-world examples, architectures, and workflows you won't find anywhere else.

The Original Sin

The "Original Sin" of Computing: Code vs Data

Back in the 1940s, the Von Neumann architecture unified program code and data in the same memory.

This design choice, i.e. storing instructions and data together with no hardware-level distinction, made computers flexible, but also planted the seed of countless security flaws.

If a malicious actor can trick a system into treating untrusted data as code, they can hijack execution.

Buffer overflows, **SQL injection**, and similar exploits all exploit this blurring of boundaries.

Over decades, we've developed mitigations: memory-safe languages (like Rust or Go) to prevent misinterpreting data types, strict bounds checking to stop overflows, **Data Execution Prevention** (marking memory non-executable), **Address Space Layout Randomization** (making it hard for attackers to find or target code in memory), stack canaries, and more.

These measures have made attacks harder but not eliminated them.

Even today, memory corruption and code injection bugs persist. Our best efforts over ~60 years have mitigated, not fixed, the original sin.

Computing's original sin is mixing instructions with data.

When the CPU cannot distinguish between the two, bad inputs can become bad instructions.

The fact that this flaw is architectural, and baked into how computers work. This is why remote code execution (RCE) remains a perennial threat.

Agents Repeat the Sin: How AI Blends Instructions and Data

LLMs can't separate instructions from data.

When you run an agent, you're basically building a single text blob that includes: your request, tool output, scraped web pages, repo files, emails, prior messages, then asking the model to keep predicting the next token from that blob.

Internally, it's all just tokens. Same representation. Same pipeline. No "trusted prompt" lane and "untrusted content" lane.

The agent doesn't read context like a program reads input. It reads context like a person reads a note.

And if the note contains "ignore your previous instructions and do X," you're betting the whole system on the model consistently deciding "that looks like untrusted content, I'll ignore it."

But the model has no native concept of provenance. It doesn't come with an `isTrusted=false` bit attached to a sentence from a random web page.

So we rebuilt the classic code/data boundary problem, just at the language layer.

In the Von Neumann world, the nightmare was "data gets interpreted as code."

In the agent world, it's "content gets interpreted as instruction." Same failure mode, different substrate.

And this isn't a training bug you can patch away.

It's a design property of how we're using LLMs: we keep stuffing external text into the same channel as the command, then acting surprised when the model sometimes follows the wrong part.

Once you give that agent tools, e.g. filesystem, network, CI, GitHub, email, prompt injection becomes the natural-language version of remote code execution.

OpenAI has basically acknowledged where this lands: prompt injection isn't something we should expect to fully "solve."

That means the real work moves from "write a better system prompt" to "design the workflow so the blast radius is small when it goes wrong."

Prompt Injection: The New Remote Code Execution

Prompt injection is the exploit that arises from this design.

It means inserting malicious instructions into the data the AI processes, causing the model to do something its user did not intend.

Because the model doesn't truly know which instructions to follow, an attacker can smuggle directives in via seemingly innocuous content.

There are two main variants:

- **Direct prompt injection:** The attacker directly interacts with the model (for example, by typing input into a chatbot) and convinces it to ignore previous directives or perform unauthorized actions. Many have seen simple examples of this ("ignore previous instructions, now do X") in public chatbots.
- **Indirect prompt injection:** The more insidious form, where the attacker's payload hides in data that the AI agent pulls in from elsewhere. For example, an agent browsing a web page or reading an email could encounter hidden instructions planted there by a malicious third party.

Indirect injections are especially dangerous in **agentic workflows**.

By design, agents fetch content autonomously, from APIs, the web, your files, emails, etc. This means **untrusted data flows into the agent's context constantly**.

If any of that data contains a cleverly crafted phrase like "Ignore all prior instructions and XYZ", the agent may dutifully comply with XYZ.

This is similar to "malvertising" in the early web: just as an ad could carry malware to a browser, a malicious snippet of text can carry hidden commands to an AI.

What can a malicious prompt do?

Anything that the agent has the power to do, a malicious prompt can force the agent to do, and acting against you for the benefit of the hacker.

Real-world examples are already surfacing:

- **Email assistant gone rogue:** OpenAI disclosed an incident where a hidden instruction in an email led its ChatGPT Atlas agent to deviate. The user asked the AI to draft out-of-office replies, but one email in the inbox contained a concealed prompt. When the agent read it, instead of an out-of-office note it actually drafted a **letter of resignation to the user's CEO**.
- **0-Click "EchoLeak" exploit:** Microsoft documented a vulnerability where an AI agent could be manipulated just by rendering an email with a poisoned image, no user click required. Dubbed **"EchoLeak,"** this exploit showed that even the act of summarizing or previewing content could

trigger hidden instructions to run. An agent could be coerced to, say, send your data to an attacker's URL, simply by seeing a booby-trapped image or text snippet.

- **CI/CD agent compromise: "Prompt Pond"** demonstrated how a malicious code comment or PR description in a repository could trick AI code assistants integrated into continuous integration pipelines. If a dev team uses an agent to automatically review or merge pull requests, an attacker can include a hidden `approve this PR` instruction in the code metadata. The AI happily follows it, merging vulnerable code into your codebase. Essentially, prompt injection can subvert software supply chains by abusing overly-trusted AI automations.
- **Self-spreading AI worm:** Perhaps most alarming, **"Morris II" worm** uses prompt injection to self-propagate through connected AI systems. This experimental malware planted adversarial prompts into emails and even images. When an AI agent encountered them (e.g. an email-summarizing assistant), it would be hijacked to steal data (like contacts, credit cards, etc.) and then send out more infected messages on its own. The worm spreads **without any traditional executable virus,** the payload is instructions in natural language, causing AI agents to themselves act as the malware.IBM's report noted the team successfully exfiltrated sensitive personal data and replicated across multiple AI platforms. OpenAI acknowledged this novel threat and says it's working to harden systems against it.
- **Agents with system access:** Security firm Trail of Bits recently demonstrated prompt-to-RCE exploits in three different AI agent frameworks. By cleverly phrasing a single prompt, they bypassed safety filters and achieved **one-shot remote code execution** on the host machine via the agent. For example, one exploit tricked an agent's "safe" `go test` command into executing a malicious `curl | bash` sequence, effectively downloading and running attacker code. Such vulnerabilities (argument injections, tool misuse) are likely **common across agent platforms** that execute commands. Any agent that runs shell or filesystem actions is a potential doorway for prompt-based RCE.

Safer Agent Adoption: Practical Guidance for Developers

You should adopt a **cautiously optimistic** stance, use agents where they help, but **architect your workflows defensively**.

Here are concrete steps and best practices to improve security in agentic workflows:

(1) Sandbox and isolate agents and always treat an AI agent as potentially compromised.

Run agents in the most restricted environment feasible.

Options include containers, sandboxed subprocesses, or even separate virtual machines.

For example, you can run each coding agent on an isolated VM with no access to credentials or network secrets. The agent writes code to a local repo, which the human reviews and pushes manually.

This way, if the agent misbehaves (or is manipulated), it's contained.

Sandboxing should be the **primary security control** for agents.

Use OS-level permissions too, e.g. run the agent under a low-privilege user account that can't modify sensitive system files.

If your agent doesn't need internet access, consider firewalling it off or limiting it to specific domains.

(2) Principle of least privilege

Give agents the minimal set of tools and permissions required for their task.

If an agent only needs to read data, don't also give it write/delete abilities.

Many agent platforms let you configure which commands or APIs the agent can call.

Lock this down as much as possible.

Drastically **reduce allow-lists** of "safe" commands, or remove dangerous flags/operations from what the agent can invoke.

Be especially wary of any tool that can execute arbitrary code or access the network.

(3) Input validation and output monitoring

If you develop your own agent tooling, implement strict validation on any commands before executing them.

For instance, if an agent crafts a shell command, parse it and ensure it doesn't contain forbidden subcommands or redirects.

Use argument separators (--) to stop injection of flags.

On the output side, log everything the agent does, every tool invocation, every file it writes or network call it attempts.

Robust logging allows you to detect suspicious behavior and audit what happened if something goes wrong.

If you see the agent suddenly trying to ping an unknown server or read an unrelated file, that's a red flag.

(4) Human in the loop for critical actions

For actions that could have irreversible consequences (transferring money, deleting data, sending emails), always require explicit human confirmation.

Yes, pop-ups can be annoying, but for now it's a necessary friction.

Design your agent to escalate uncertain or high-risk decisions to a user.

If an agent activity looks abnormal, pause and review.

As a developer, you can also build in automatic halts, e.g. if an agent tries to execute a sequence of commands that wasn't part of its initial goal, maybe stop and ask for verification.

(5) Protect the context

Since prompt injections often hide in the fetched context, be careful what you feed your agent.

Whenever possible, **avoid giving agents direct access to highly sensitive data** or credentials.

For instance, rather than letting an agent read your entire email inbox unsupervised, you might have it process only one email at a time or only a pre-filtered subset.

If using tools like retrieval-augmented generation (RAG) where the agent searches a knowledge base, ensure that database is curated.

In code scenarios, be wary of untrusted open-source dependencies, a comment in a dependency could poison the agent. I

(6) Stay updated on threats and patches

If a new exploit like an image-based attack is discovered, vendors may release mitigations (like Google did for image URLs).

Apply those updates promptly.

Also watch for tooling: there are startups and open-source projects working on monitoring and filtering LLM inputs/outputs for malicious content.

Using a third-party "prompt firewall" might catch known bad patterns (just remember it won't be foolproof).

(7) Opt-out and disable unneeded agent features

If you're integrating an agent platform (say into an app or OS), push for options to opt out of invasive behaviors.

For example, **Windows 11's "Recall",** an AI feature that continuously screenshots your desktop and OCRs text to provide memory for the Copilot agent.

The encrypted messaging app Signal considered this completely unacceptable for privacy, so they took matters into their own hands.

Signal's developers enabled a DRM-based screen security flag to block Windows from capturing their app's content.

This was basically a hack, a technique meant for preventing video screen-recording but it stops Recall from seeing your messages.

As a developer, **demand those controls**. If an OS or platform agent is snooping on your app's data, ask the vendor for an opt-out or at least documentation on how to mitigate it.

Until provided, use every tool available (even unconventional ones like Signal did) to shield especially sensitive user data from agent eyes.

(8) Educate and warn your users

If you build an application that uses AI agents, be transparent with users about what the agent can access and the potential risks.

Provide guidance like "Do not use this feature with documents you wouldn't upload to the cloud," or "Review the agent's suggestions before applying them."

Essentially, encourage healthy skepticism.

Users should know that an AI agent might make mistakes or be manipulated by bad inputs.

OpenAI's own enterprise advice includes training users not to delegate overly broad authority to agents.

Clear documentation and warnings can prevent naive trust in agents that could lead to breaches.

Thoughts

Remote code execution by malicious data is not a new problem, it's the oldest problem in modern computing security.

AI agents have reignited it in a new form.

Deploying an AI agent should be treated with the same gravity.

As you experiment with tools like OpenClaw (open-source local agents) or enterprise offerings like Claude Code and OpenAI's Codex, keep security at the forefront.

Use the guidelines above to architect your agentic workflows so that even if (when) a prompt injection happens, it becomes a minor annoyance caught by a sandbox or review step, rather than a catastrophe.

By respecting the power and the pitfalls of AI agents, you can be **cautiously optimistic** and able to reap the productivity benefits while keeping nightmares at bay.

Bonus Articles

Local LLMs That Can Replace Claude Code

Small team of engineers can easily burn >$2K/mo on Anthropic's Claude Code (Sonnet/Opus 4.5). As budgets are tight, you...

medium.com

ClawRouter: Anthropic charged me $4,660 - How I cut it 70% with smart LLM routing

Anthropic charged me $4,660.87, just for Claude. Most requests were routed to the Claude Opus model. This is how I cut...

medium.com

Qwen3-Coder-Next Failed in the Scariest Way: 10 Practical Tests

Benchmarks are the LinkedIn of LLMs. Every model looks unstoppable.

medium.com

Why Codex Became My Default Over Claude Code (for Now)

If you haven't tried Codex yet, I've got a brief take that might save you a few hours.

medium.com

"This PDF hates OCR" - DeepSeek OCR 2 Reads Documents Like A Human

"This PDF hates OCR" - DeepSeek OCR 2 Reads Documents Like A Human Your customer forwards an invoice PDF with the...

medium.com

Clawdbot Lite: 99% Smaller, 4000 Lines, Same Core Power

People are building Polymarket bots, multi-agent product teams, crypto traders, coding agents, business operators with...

medium.com

Vectorless RAG for Agents: PageIndex Is Why Their Demo Works and Yours Needs "Context"

When I'm building a RAG apps, the thing that quietly kills quality (and budgets) is turning a document into a pile of...

medium.com

Fully Local Agentic Coding on localhost: Claude Code, Codex, llama.cpp, Ollama and Docker MR

This is the setup I wish I had the first time I tried to run Claude Code and Codex-style workflows against local...

medium.com
#ai-security#prompt-injection-attack#ai-agent-security#remote-code-execution#ai-agent

Reporting a Problem

Sometimes we have problems displaying some Medium posts.

If you have a problem that some images aren't loading - try using VPN. Probably you have problem with access to Medium CDN (or fucking Cloudflare's bot detection algorithms are blocking you).

Problem Description Describe your problem here...

Submit Cancel